

UVMF YAML

Structure and Format Reference Manual

User Guide — Complete & Practical

Primary technical basis: UVMF 2023.4

Practical flow examples cross-checked with UVMF 2019.4 tutorial/workbook material

Prepared as an engineering desk reference for YAML-driven UVMF code generation

Document Control

Item	Value
Document	UVMF YAML Structure and Format Reference Manual — User Guide
Edition	Version 1.1
Primary release basis	UVMF YAML Reference Manual 2023.4 and UVM Framework Users Guide 2023.4
Practical flow basis	UVMF ALU Tutorial / Student Workbook 2019.4, used only where consistent or explicitly labeled legacy
Purpose	Provide a self-contained, practical reference for creating, generating, regenerating, debugging, and running UVMF YAML-based verification configurations and VRM regression test lists.
Scope	Interfaces, utility components, environments, benches, global data, generator command flow, merge flow, generated output, VRM YAML regression test lists, worked examples, and troubleshooting.

Version rule: Treat the installed UVMF release and its generated templates as the final authority. This guide uses UVMF 2023.4 as the primary schema reference. Older 2019.4 tutorial material is clearly identified when it exposes a legacy key or behavior.

Source Basis

- UVMF YAML Reference Manual, Version 2023.4 — formal YAML hierarchy, schemas, generator options, merge behavior, and global data structures.
- UVM Framework Users Guide, Version 2023.4 - practical code-generation flow, generated file responsibilities, interface/environment/bench customization points, QVIP integration context, and VRM YAML regression test-list format.
- UVM Framework ALU Tutorial: A Step By Step Guide, UVMF 2019.4 — practical decomposition of interface, environment, predictor, scoreboard, and bench YAML.
- UVM Framework Student Workbook, 2019.4 — end-to-end workflow framing: plan stimulus/checks, create interfaces, environment, bench, complete bench, start verification.

Contents

Part I — Foundations

1. Purpose, Scope, and How to Use This Manual	4
2. UVMF Generation Model	5
3. YAML Syntax Rules for UVMF	6
4. UVMF Top-Level YAML Hierarchy	8
5. Running yaml2uvmf.py	9
6. Regeneration and Merge Workflow	11

Part II — YAML Structure Reference

7. Interface YAML Structure	12
8. Utility Component YAML Structure	16
9. Environment YAML Structure	17
10. Bench YAML Structure	22
11. Global YAML Structure	24

Part III — Practical Engineering Workflow

12. Multi-File Organization and Dependency Rules	25
--	----

13. From DUT Specification to YAML	26
14. Generated Output and Where to Customize	27
15. Worked ALU-Style Example	28
16. Practical AXI4-Stream-Style Interface Template	30
17. Validation, Debugging, and Common Errors	31
18. Quick Reference Checklists	33
19. YAML Test List Format for UVMF VRM	35
Appendix A - Complete YAML Skeletons	38
Appendix B - Schema Quick Reference	40
Appendix C - Release and Documentation Notes	42
Appendix D - Glossary	43
Appendix E - Source Documents Used	44

1. Purpose, Scope, and How to Use This Manual

This manual is intended to be used while writing or reviewing UVMF YAML, not only as background reading. It combines the formal 2023.4 schema descriptions with a practical workflow for moving from a DUT interface definition to generated UVMF code, compiling the generated output, and then adding protocol- or DUT-specific logic in the intended customization locations.

1.1 What this guide covers

- The required UVMF YAML root structure and how multiple YAML files compose into one generation model.
- Interface YAML: clocks, reset, ports, transactions, configuration variables, constraints, parameters, typedefs, and DPI definitions.
- Utility-component YAML: predictor, coverage, and custom scoreboard shells.
- Environment YAML: agents, sub-environments, scoreboards, TLM connectivity, configuration propagation, register models, QVIP integration, and typedefs.
- Bench YAML: top environment, BFM active/passive mode, clocks/resets, BFM parameters, imports, plusargs, and additional top modules.
- Global YAML: output paths, common source headers, flat output, and automatic BFM parameter elaboration.
- Generator command usage, regeneration/merge behavior, generated output, validation, and troubleshooting.
 - VRM YAML test lists: regression testbench selection, repeat counts, seeds, UVM test mapping, per-test runtime arguments, includes, and symbolic links.

1.2 Recommended reading paths

If you need to...	Go to...
Create a new protocol agent	Chapters 3, 4, 7, then Chapter 14
Create a block-level environment	Chapters 8–9, then Chapters 12–14
Generate a complete testbench	Chapters 9–10 and Chapter 5
Regenerate after YAML changes	Chapter 6
Parameterize BFMs or environments	Chapters 7, 9, 10, and 11
Connect predictors and scoreboards	Chapter 9 and the data-flow example in Chapter 15
Debug YAML/generator issues	Chapter 17
Find a key quickly	Appendix B
Create or review a VRM YAML regression test list	Chapter 19

2. UVMF Generation Model

UVMF uses configuration data to generate reusable verification infrastructure. The YAML-based flow describes hierarchy and metadata; the generator expands that description into SystemVerilog/UVM packages, classes, BFM, modules, file lists, and Makefiles. The generated code is intended to simulate as generated, giving the engineer a structurally valid starting point before protocol-specific behavior is added.

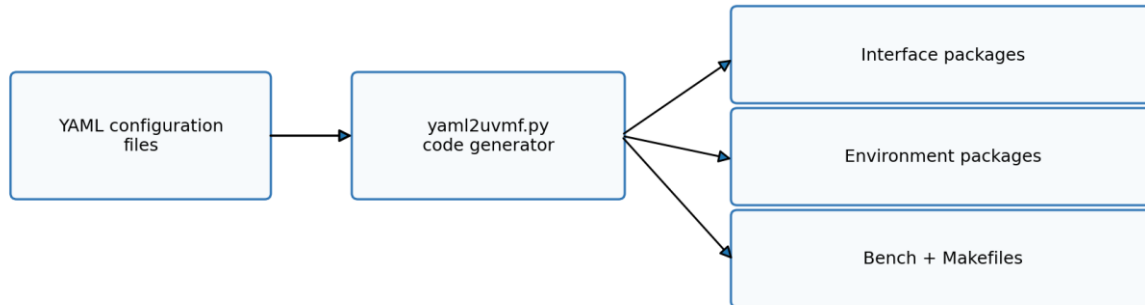


Figure 2-1. YAML-driven UVMF generation flow.

2.1 What is generated at each level

YAML level	Representative generated content	Engineering intent
Interface	Transaction class, sequence base/random sequence, coverage, driver, monitor, agent, agent configuration, reg adapter/predictor support, driver/monitor BFM, package, file list, Makefile	Reusable protocol-level verification IP.
Utility component	Predictor, coverage component, or custom scoreboard shell with declared analysis ports/exports	Analysis logic that sits inside an environment.
Environment	Environment class/configuration, sequence base, analysis-component instances, scoreboards, TLM connections, environment package, file list, Makefile	Reusable block/subsystem verification environment.
Bench	Top test, virtual sequence, bench/sequence/parameter packages, hdl_top, hvl_top, simulation files and Makefile	DUT-specific top-level testbench assembly.

2.2 Hierarchical dependency rule

When generating a higher level, provide the YAML definitions for the UVMF objects underneath it. For example, an environment generation run must also receive the YAML for referenced interfaces and generator-defined utility components; a bench generation run must receive the YAML for the top environment and recursively referenced UVMF structures.

Practical rule: Keep a repeatable file list for each top-level bench. If one command line can reproduce the entire hierarchy from a clean destination directory, the YAML set is much easier to review and maintain.

3. YAML Syntax Rules for UVMF

YAML is indentation-sensitive. UVMF uses it as a compact serialization format for nested verification data structures. A syntactically valid YAML file can still be invalid for UVMF if required keys, schema fields, or referenced component definitions are missing, so validation must include the UVMF generator itself.

3.1 Core formatting rules

Rule	Practical guidance
Indentation	Use spaces consistently. Do not use tab characters for YAML indentation. Two spaces per nesting level is a practical convention.
Mappings	Use key/value form: key: value. Nested content is indented beneath its parent key.
Lists	Use a dash for each ordered list item. List order is preserved in generated output where the reference manual states list ordering is maintained.
Comments	Start a comment with #. It continues to end of line.
Strings	Quote values when they contain SystemVerilog syntax, punctuation, percent format specifiers, or values that YAML could reinterpret.
Boolean-like UVMF values	UVMF examples frequently use quoted "True" and "False" strings. Follow the style of the installed release templates.
Block strings	Use YAML block scalar for multi-line text such as the global source header.
File organization	The UVMF hierarchy may be described in one YAML file or spread across multiple YAML files.

3.2 Mapping and list examples

Nested mapping

```

uvmf:
  interfaces:
    simple_if:
      clock: clk
      reset: rst_n
      reset_assertion_level: "False"

```

Ordered list of schema objects

```

parameters:
- name: "DATA_WIDTH"
  type: "int"
  value: "32"
- name: "ID_WIDTH"
  type: "int"
  value: "4"

```

3.3 SystemVerilog text inside YAML

Several UVMF fields intentionally carry SystemVerilog fragments: transaction types, port widths, constraint bodies, typedef definitions, register-adaptor types, and parameter expressions. Treat these strings as source text. Quoting makes the boundary between YAML syntax and SystemVerilog syntax explicit.

```

transaction_constraints:
- name: "word_aligned_c"
  value: "{ address[1:0] == 2'b00; }"
  comment: "Generate word-aligned addresses."

```

3.4 Common YAML mistakes

- Mixing tabs and spaces or changing indentation depth in the middle of a mapping.
- Writing a list item without the leading dash.
- Placing a property at the wrong hierarchy level, for example bench properties under environments.
- Unquoted SystemVerilog literals or expressions that YAML interprets unexpectedly.
- Misspelling schema field names such as receiver, trans_type, iscompare, or reset_assertion_level.
- Using a referenced agent/environment/utility component type without providing its generator definition in the same run when required.

3.5 Pre-Generation YAML Review Checklist

Before invoking `yaml2uvmf.py`, perform this structural review. It catches the most common YAML and UVMF-model errors before generated-code debugging begins.

- Confirm the document is rooted under `uvmf:` and each definition is under the correct interfaces, `util_components`, `environments`, `benches`, or `global` section.
- Use spaces only for indentation; do not use tab characters.
- Confirm every referenced interface, utility-component, and sub-environment definition required by the generation target is included in the same generator run.
- Check every port direction from the testbench/BFM point of view, not from the DUT point of view.
- Review transaction fields individually: use `isrand` only for stimulus fields that may be randomized, and `iscompare` only for fields with meaningful comparison semantics.
- Keep transaction content separate from timing metadata and from persistent agent/environment configuration variables.
- Quote SystemVerilog fragments, width expressions, and release-style `True/False` values consistently with the installed UVMF templates.
- Validate names used in TLM paths, configuration paths, BFM active/passive overrides, and parameter overrides against the instantiated component names.
- Generate into a clean destination while the YAML is still changing structurally so skipped existing files cannot hide a change.
- Compile or run the generated structural bench before adding protocol-specific driver, monitor, predictor, scoreboard, or DUT code.

Minimum validation loop

1. Parse the YAML successfully.
2. Run `yaml2uvmf.py` and resolve schema/reference errors.
3. Compile the generated output without DUT-specific hand edits.
4. Add DUT/protocol customization only in intended generated-code locations.
5. After structural YAML changes, regenerate/merge and compile again before adding new manual edits.

4. UVMF Top-Level YAML Hierarchy

All UVMF YAML is rooted under the `uvmf:` key. Under that root, the 2023.4 reference defines interfaces, `util_components`, environments, benches, and global data. These sections can coexist in one file or be distributed across several files passed to the same generation run.

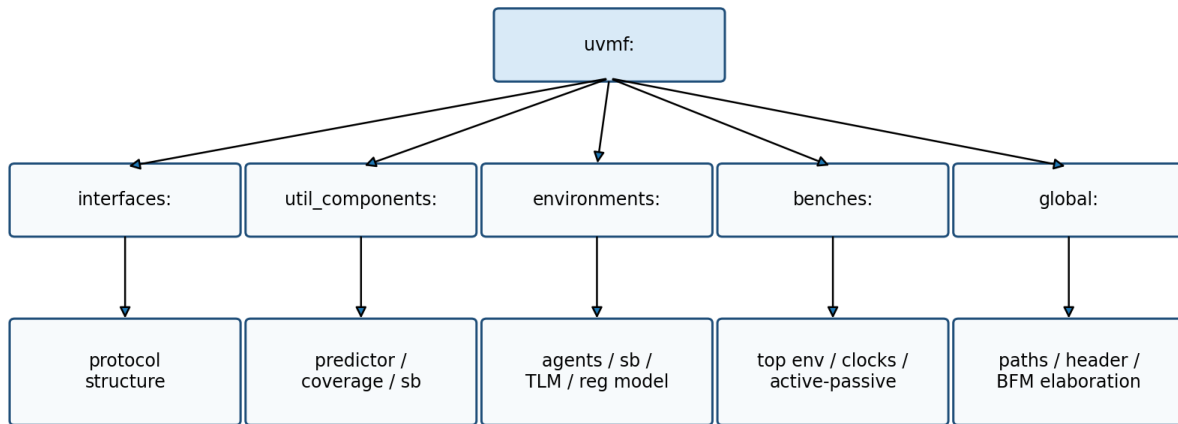


Figure 4-1. UVMF top-level YAML organization.

Master top-level pattern

```

uvmf:
  interfaces:
    <interface_name>:
      <interface properties>

  util_components:
    <utility_component_name>:
      <utility properties>

  environments:
    <environment_name>:
      <environment properties>

  benches:
    <bench_name>:
      <bench properties>

  global:
    <global properties>
  
```

4.1 Naming model

The name under each section is the generator-visible logical name of that object. Environment component schemas then use a separate name for the instance and type for the definition being instantiated. Keeping this distinction clear avoids many path and connectivity mistakes.

Context	name means...	type means...
interfaces: my_bus:	Definition name of the interface	Not applicable at definition level
agents: - name: req_agent, type: my_bus	Environment instance name	Interface definition name
analysis_components:	Environment instance name	Utility-component definition name
subenvs:	Sub-environment instance name	Environment definition name

5. Running yaml2uvmf.py

5.1 Basic command form

```
yaml2uvmf.py [options] [yaml_file1 [yaml_file2 ... yaml_fileN]]
```

In many installations the script is invoked through Python, for example:

```
python $UVMF_HOME/scripts/yaml2uvmf.py interface.yaml environment.yaml bench.yaml
```

5.2 Recommended initial-generation sequence

1. Sketch the intended interface/environment/bench hierarchy first.
2. Translate the hierarchy into YAML structures.
3. Run yaml2uvmf.py against the YAML set.
4. Build or run the generated output before making manual source edits. The reference flow recommends make cli as an early validation point.
5. If generation/build errors occur at this stage, correct the YAML and regenerate rather than patching generated source.
6. Only after the generated structure is sound should protocol- and DUT-specific edits be added in designated customization areas.

5.3 Common command switches

Switch	Purpose / practical note
--version	Print generator version. Use this when correlating behavior with documentation.
-h, --help	Print the script help. Treat this as the final authority for the installed release.
-q, --quiet	Reduce generator console output.
-d, --dest_dir	Change the generation destination; default is typically ./uvmf_template_output.
-o, --overwrite	Overwrite existing generated output. Without this, generation normally skips existing files.
-f, --file	Read a list of YAML configuration files from a file.
-F, --refile	Users Guide 2023.4 documents a relative file-list form whose relative paths are resolved from the list file location.
-g, --generate	YAML Reference 2023.4 documents generation of only a named component. See the documentation discrepancy note below.
-t, --template_dir	Override the template source location.
-c, --clean	YAML Reference 2023.4 documents cleanup of generated code instead of generation.
--no_archive_yaml	Users Guide 2023.4 documents disabling the archived YAML output directory.
-m, --merge_source	Enable merge/regeneration using an existing hand-edited source tree.
-s, --merge_skip_missing_blocks	Continue when a labeled block from old source cannot be mapped; a report is produced.
--merge_no_backup	Disable default backup behavior during merge.
--merge_debug	Keep intermediate newly generated output for merge debugging.
--merge_verbose	Print detailed information about mapped files and named blocks.

Documentation discrepancy: The 2023.4 YAML Reference lists -g for --generate. The 2023.4 Users Guide command table appears to print -f for --generate even though -f is already used for --file. Run yaml2uvmf.py --help in the installed release and use the script-reported switch.

5.4 Output strategy during development

During first-pass YAML development, generate into a disposable clean directory. This avoids confusion caused by the default "skip existing files" behavior and makes it obvious whether a YAML change actually changed generated content. Once manual edits exist, transition to the merge flow described in Chapter 6.

6. Regeneration and Merge Workflow

UVMF 2023.4 supports regeneration after manual edits by transferring content contained in labeled UVMF pragma blocks from old generated source into newly generated output. This is the safe mechanism for evolving the YAML model without discarding intended customization.

6.1 Custom block form

```
// pragma uvmf custom <label_name> begin
// hand-written code here
// pragma uvmf custom <label_name> end
```

6.2 Merge behavior

Situation	Merge behavior
Manual edit inside a matching UVMF pragma block	Transferred into the corresponding block in newly generated output.
File created manually in the old source tree	Left alone.
File exists only in new generation	Copied into the source tree.
Manual edit outside UVMF pragma blocks	Not preserved; the reference warns these edits can be lost without a warning.
Custom pragma block exists only in old source	Not automatically transferred. Default merge behavior flags missing blocks unless skip option is used.

6.3 Safe regeneration sequence

- Commit or otherwise back up the working source tree.
- Update YAML only for structural changes.
- Generate/merge with `--merge_source` pointing at the hand-edited source.
- Use `--merge_debug` when you need an untouched representation of what the new YAML alone produces.
- Use `--merge_verbose` to see mapped blocks and copied files.
- Compile immediately after merge. YAML changes can invalidate hand-written references even when the merge itself succeeds.

High-risk case: If a transaction variable is removed or renamed in YAML, hand-written driver, monitor, predictor, scoreboard, or sequence code may still reference the old field. Merge success therefore does not imply compile success.

7. Interface YAML Structure

An interface YAML definition captures the protocol-level structure used to generate the reusable UVMF agent package. At minimum the 2023.4 reference requires a primary clock and reset. A useful interface normally also defines ports, transaction variables, configuration data, and any parameterization needed by the generated classes/BFMs.

7.1 Minimal interface skeleton

```

uvmf:
  interfaces:
    simple_if:
      clock: clk
      reset: rst_n
      reset_assertion_level: "False"
      ports:
        - name: req
          width: "1"
          dir: "output"
      transaction_vars:
        - name: req
          type: "bit"
          isrand: "True"
          iscompare: "True"

```

7.2 Core top-level properties

Property	Type	Meaning / default
existing_library_component	string	Marks the interface as already generated/reference-only unless overwrite behavior is requested.
clock	string	Required primary clock name. Additional clocks are manual additions.
reset	string	Required primary reset name. For a no-reset interface, the reference suggests a dummy reset plus manual cleanup of generated reset code.
reset_assertion_level	True False	True means active-high reset; False means active-low.
vip_lib_env_variable	string	Environment variable pointing to source location for the interface package.
veloce_ready	True False	Defaults True; controls generation of Veloce-ready/friendly interface code.
mtlb_ready	True False	Defaults False; enables Matlab-oriented generated files.
config_constraints	list(constraint_schema)	Constraints applied to configuration variables.
dpi_define	dpi_schema	DPI-C source definition associated with the interface.

7.3 Common schema-backed interface sections

The 2023.4 interface chapter defines schemas used by common interface fields. The examples and the 2019.4 tutorial show the practical field names below. Follow the installed new_interface.yaml template for your exact release.

Field	Schema / contents	Use
imports	import_schema	Import packages needed by generated interface package code.
parameters	parameter_def_schema	Define class/BFM parameters such as address or data widths.
hdl_typedefs	typedef_schema	Types needed on HDL/BFM side.
hvl_typedefs	typedef_schema (shown in 2019.4 tutorial)	Types needed on HVL/class side.
ports	port_schema	Define signal bundle ports. Direction is from the testbench/BFM perspective in the practical tutorial.
transaction_vars	transaction_schema	Untimed transaction fields carried by sequence items.

Field	Schema / contents	Use
config_vars	config_var_schema	Agent configuration object variables.
transaction_constraints	constraint_schema	SystemVerilog constraints applied to transaction variables.

7.4 import_schema

Field	Required	Meaning
name	Yes	Package name to import.

```
imports:
- name: "protocol_types_pkg"
- name: "common_verif_pkg"
```

7.5 parameter_def_schema

Field	Required	Meaning
name	Yes	Parameter identifier.
type	Yes	SystemVerilog type, such as int.
value	No	Default parameter value. Omit to declare without a default.

```
parameters:
- name: "DATA_WIDTH"
  type: "int"
  value: "32"
```

7.6 typedef_schema

Field	Required	Meaning
name	Yes	Typedef name.
type	Yes	SystemVerilog type text.

```
hdl_typedefs:
- name: "opcode_t"
  type: "enum bit [2:0] {NOP, ADD, XOR, MUL}"
```

7.7 port_schema

Field	Required	Meaning
name	Yes	Signal name.
width	Yes	Scalar width string or list of width strings for a multi-dimensional packed array. Values can be numeric or parameter expressions.
dir	Yes	input, output, or inout. In the practical UVMF tutorial, direction is relative to the testbench.
reset_value	No	Value driven on reset; reference states default 0 when omitted.

```
ports:
- name: "wdata"
  width: "DATA_WIDTH"
  dir: "output"
- name: "ready"
  width: "1"
  dir: "input"
- name: "packed_matrix"
  width: ["15", "3", "DATA_WIDTH"]
  dir: "output"
```

Direction convention: For an initiator BFM driving DUT inputs, command/data outputs from the BFM are declared dir: output; DUT handshake/status signals observed by the BFM are declared dir: input.

7.8 transaction_schema

Field	Required	Default / behavior
name	Yes	Transaction field name.
type	Yes	SystemVerilog type text.
isrand	No	Default False. True adds rand so object randomize() can modify the field.
iscompare	No	Default True. False excludes metadata such as latency/timestamps from transaction comparison.
unpacked_dimension	No	Text placed to the right of the variable name to create an unpacked dimension.
comment	No	Emitted above the declaration.

```
transaction_vars:
- name: "data"
  type: "bit [31:0]"
  isrand: "True"
  iscompare: "True"
- name: "transfer_delay"
  type: "int unsigned"
  isrand: "True"
  iscompare: "False"
comment: "Inter-transfer timing metadata."
```

7.9 config_var_schema

Field	Required	Default / behavior
name	Yes	Configuration variable name.
type	Yes	SystemVerilog type.
isrand	No	Default False; True adds rand.
value	No	Initial value at beginning of simulation.
comment	No	Generated declaration comment.

```
config_vars:
- name: "max_wait_cycles"
  type: "int unsigned"
  isrand: "True"
  value: "16"
comment: "Maximum legal backpressure interval."
```

7.10 constraint_schema

Field	Required	Meaning
name	Yes	Constraint name.
value	Yes	SystemVerilog constraint body.
comment	No	Generated comment above constraint.

```
transaction_constraints:
- name: "legal_len_c"
  value: "{ length inside {[1:256]}; }"
comment: "Limit generated transaction length."
```

7.11 DPI schemas

The interface `dpi_define` structure can declare a shared-object name, C source files, compile/link arguments, and DPI imports. The 2023.4 reference explicitly notes that DPI exports are currently unsupported.

dpi_define field	Purpose
name	Shared-object name.
files	Array of C source files.

dpi_define field	Purpose
comp_args	C compile arguments.
link_args	C link arguments.
imports	Array of dpi_import_schema entries.
exports	Documented in structure, but the 2023.4 reference states DPI exports are unsupported.

dpi_import field	Purpose
name	Imported C function name.
return_type	C/SystemVerilog DPI return type.
c_args	C argument-string text.
sv_args	Argument definitions: name, type, dir, optional unpacked_dimension.

7.12 Interface YAML design checklist

- Does every physical interface signal belong in a logical UVMF interface/agent?
- Is port direction written from the BFM/testbench point of view?
- Which values are real transaction content versus timing/metadata?
- Should each transaction field be randomized? compared?
- Are widths literal or parameterized?
- Are configuration knobs agent configuration variables rather than transaction fields?
- Can all constraints be expressed at the transaction/configuration level without protocol timing code?
- Does the interface need register-model adapter support or DPI integration?

8. Utility Component YAML Structure

Utility components are environment-level analysis components that are neither agents nor sub-environments. UVMF 2023.4 supports three utility-component types: predictor, coverage, and scoreboard.

8.1 Top-level properties

Property	Type	Meaning
type	predictor coverage scoreboard	Selects generated utility-component form.
analysis_exports	list(analysis_schema)	Analysis imp/export inputs receiving transactions.
analysis_ports	list(analysis_schema)	Analysis ports broadcasting transactions.
qvip_analysis_exports	list(analysis_schema)	Analysis imps that accept mvc_sequence_item_base and cast to the specified transaction type for QVIP-facing connectivity.
parameters	list(parameter_def_schema)	Utility-component parameter definitions.

8.2 analysis_schema

Field	Required	Meaning
name	Yes	Port/export name.
type	Yes	Sequence item / transaction type handled by the analysis connection.

```

uvmf:
  util_components:
    my_predictor:
      type: predictor
      analysis_exports:
        - name: "req_ae"
          type: "req_transaction"
      analysis_ports:
        - name: "predicted_ap"
          type: "rsp_transaction"

```

8.3 Generated behavior by utility type

Type	Generated starting point	What the engineer must add
predictor	Analysis exports/ports and a transaction broadcast path	Prediction/reference-model logic in generated write functions. The reference notes generated placeholder broadcasting is for connection validation and can leave scoreboard residue until replaced.
coverage	Coverage component, covergroup, environment configuration handle	Coverpoints, bins, crosses, exclusions, and sampling policy.
scoreboard	Custom scoreboard shell with selected analysis ports/exports and write functions	Storage, matching policy, comparison, ordering rules, error reporting.

9. Environment YAML Structure

Environment YAML describes composition: which UVMF agents, non-UVMF components, utility components, built-in scoreboards, sub-environments, QVIP sub-environments, and register-model structures are instantiated, how they are parameterized/configured, and how transaction-level connections are wired.

9.1 Practical environment skeleton

```

uvmf:
  environments:
    block_env:
      agents:
        - name: "req_agent"
          type: "req_if"
        - name: "rsp_agent"
          type: "rsp_if"

      analysis_components:
        - name: "pred"
          type: "block_predictor"

      scoreboards:
        - name: "rsp_sb"
          sb_type: "uvmf_in_order_scoreboard"
          trans_type: "rsp_transaction"

      tlm_connections:
        - driver: "req_agent.monitored_ap"
          receiver: "pred.req_ae"
          validate: "True"
        - driver: "pred.predicted_ap"
          receiver: "rsp_sb.expected_analysis_export"
          validate: "True"
        - driver: "rsp_agent.monitored_ap"
          receiver: "rsp_sb.actual_analysis_export"
          validate: "True"

```

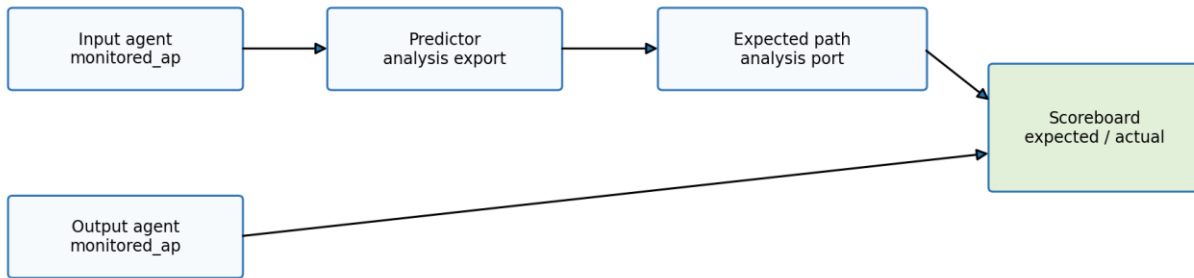


Figure 9-1. Typical agent → predictor → scoreboard transaction flow.

9.2 Top-level environment properties

Property	Type	Meaning
existing_library_component	string	Reference-only already generated environment unless overwrite is used.
agents	list(component_schema)	UVMF interface agents instantiated in the environment. Built-in agent analysis port is monitored_ap.
non_uvmf_components	list(component_schema)	Components not defined by the UVMF generator.
qvip_memory_agents	list(component_schema)	Agents associated with QVIP DDR memory models.
parameters	list(parameter_def_schema)	Environment class/package parameter definitions.
hvl_pkg_parameters	list(parameter_def_schema)	Parameters included in interfaces_pkg package declaration.
imports	list(import_schema)	Packages imported by the environment package.

Property	Type	Meaning
analysis_components	list(component_schema)	Predictor/coverage/custom utility components instantiated in the environment.
scoreboards	list(scoreboard_schema)	Built-in UVMF scoreboards; expected_analysis_export and actual_analysis_export are the standard analysis exports.
subenvs	list(component_schema)	Generator-defined sub-environments.
qvip_subenvs	list(qvip_env_schema)	QVIP Configurator-generated sub-environments.
analysis_ports	list(tlm_port_schema)	Environment-level analysis ports and connectivity.
analysis_exports	list(tlm_port_schema)	Environment-level analysis exports and connectivity.
tlm_connections	list(tlm_schema)	Point-to-point UVMF TLM connections.
qvip_connections	list(qvip_tlm_schema)	Connections driven by QVIP analysis ports.
config_vars	list(config_var_schema)	Environment configuration variables.
config_variable_values	list(config_value_schema)	Default values pushed into underlying agent/sub-environment configurations using dot paths.
config_constraints	list(constraint_schema)	Constraints on environment configuration variables.
imp_decls	list(imp_decl_schema)	Names of analysis imp_decl macros to generate.
register_model	reg_model_schema	Register-model instantiation/connection configuration.
dpi_define	dpi_define_schema	DPI-C source associated with environment; structure matches interface form.
typedefs	typedef_schema	Environment typedef definitions.
mtlb_ready	True False	Defaults False; enables Matlab-oriented generated files.

9.3 component_schema

Field	Meaning
name	Instance name in this environment.
type	Definition/class type. For agents this is the interface definition name; for subenvs it is the environment definition name.
extdef	True/False. When True, allows instantiation of a component defined externally rather than by YAML in the current generator run.
parameters	Optional list of parameter_use_schema name/value overrides.
qvip_environment	Shown in the 2023.4 qvip_memory_agents example to identify the containing configurator-generated QVIP environment.
reg_block_instance_name	Shown in the 2023.4 subenv example for register-model hierarchy integration.

```
agents:
- name: "control_plane"
  type: "mem_if"
  parameters:
  - name: "ADDR_WIDTH"
    value: "CP_ADDR_WIDTH"
  - name: "DATA_WIDTH"
    value: "CP_DATA_WIDTH"
```

9.4 scoreboard_schema

Field	Meaning
name	Scoreboard instance name.
sb_type	UVMF scoreboard class type.
trans_type	Transaction type compared by the scoreboard.
parameters	Optional parameter overrides.

```
scoreboards:
- name: "rsp_sb"
  sb_type: "uvmf_in_order_scoreboard"
  trans_type: "rsp_transaction"
```

9.5 parameter schemas and imports

`parameter_use_schema` is a name/value pair used to override a parameter on an instantiated component. `parameter_def_schema` defines a new parameter using name, type, and optional default value. `import_schema` supplies one package name per entry.

```
parameters:
- name: "NUM_CHANNELS"
  type: "int"
  value: "4"

imports:
- name: "common_types_pkg"
```

9.6 qvip_env_schema

Field	Meaning
name	QVIP sub-environment instance name.
type	QVIP Configurator-generated environment type.

9.7 tlm_port_schema

Field	Meaning
name	Environment analysis port/export name.
trans_type	Transaction type parameterizing the port/export.
connected_to	Component endpoint that drives or consumes the items.

9.8 tlm_schema

Field	Meaning
driver	Port that emits transactions.
receiver	Export/imp that consumes transactions.
validate	Optional True/False check against the generator's known valid endpoints.

Built-in endpoint names: A UVMF agent publishes monitored transactions through `monitored_ap`. Built-in UVMF scoreboards expose `expected_analysis_export` and `actual_analysis_export`. These names make standard predictor/scoreboard connectivity concise.

9.9 qvip_tlm_schema

Field	Meaning
driver	QVIP agent path under its QVIP sub-environment.
ap_key	String key selecting a QVIP analysis port from the agent analysis-port array.
receiver	Receiving export/imp.
validate	Optional endpoint validation switch.

9.10 config_var_schema and config_value_schema

Environment `config_var_schema` mirrors the interface configuration-variable form: name and type are required; `isrand`, `value`, and `comment` are optional. `config_variable_values` is different: it assigns values into underlying configuration objects using dot-separated paths relative to the current environment configuration object.

```

config_variable_values:
  - name: "subenv.agent_cfg.max_wait_cycles"
    value: "24"
  - name: "req_agent.string_mode"
    value: "fast"

```

Path correctness: The 2023.4 reference warns that incorrect configuration paths or incompatible value types can become compile-time or runtime simulation errors.

9.11 imp_decl_schema

```

imp_decls:
  - name: "REQ_EXPECTED"
  - name: "REQ_ACTUAL"

```

9.12 register_model schema

The environment register_model section controls register-model instantiation, optional adapter use, explicit prediction, package/class names, and the interface used by the map. The 2023.4 reference states that only one map entry is supported; map entries after the first are ignored.

Field	Meaning
use_adapter	True/False; defaults True in the 2023.4 description.
use_explicit_prediction	True/False; defaults True.
reg_model_package	Package containing the register model.
reg_block_class	Top register block class.
reg_adapter_class	Adapter class, especially relevant for qvip/other interface types.
maps[].name	Register-map name.
maps[].interface	UVMF agent, QVIP path, or custom interface reference.
maps[].interface_type	uvmf qvip other.
maps[].qvip_agent	Legacy/alternate QVIP indication; defaults False if omitted. The reference says interface_type overrides it when present.

```

register_model:
  use_adapter: "True"
  use_explicit_prediction: "True"
  reg_model_package: "reg_model_pkg"
  reg_block_class: "reg_block"
  reg_adapter_class: "my_reg_adapter"
  maps:
    - name: "bus_map"
      interface: "control_plane"
      interface_type: "uvmf"
      qvip_agent: "False"

```

9.13 Environment design checklist

- List all agents and decide whether each is reusable protocol VIP or a one-off external component.
- Identify every monitored transaction stream and who consumes it.
- For each scoreboard, identify the expected source and actual source explicitly.
- Use utility-component predictors/coverage only when their definitions are supplied to generation.
- Decide whether block-level environments must be reusable as sub-environments at chip level.
- Define parameter propagation and configuration defaults before writing sequences.
- If using QVIP, confirm sub-environment names, agent paths, AP keys, and transaction types from the QVIP documentation.
- If using a register model, confirm adapter/predictor strategy and the one-map limitation in the referenced 2023.4 schema.

10. Bench YAML Structure

Bench YAML describes the DUT-specific top level: which environment to instantiate, how to configure BFM activity, how the generated bench drives clock/reset, how BFM parameters are elaborated, and which extra top modules or plusargs are included.

10.1 Minimal practical bench

```
uvmf:
  benches:
    block_tb:
      top_env: "block_env"
      clock_half_period: "5ns"
      clock_phase_offset: "0ns"
      reset_assertion_level: "False"
      reset_duration: "100ns"
      active_passive:
        - bfm_name: "req_agent"
          value: "ACTIVE"
        - bfm_name: "rsp_agent"
          value: "PASSIVE"
```

10.2 Bench top-level properties

Property	Type	Meaning
existing_library_component	string	Reference-only already generated bench unless overwrite is requested.
top_env	string	Required top-level environment name; its YAML definition must be available.
top_env_params	list(parameter_use_schema)	Parameter overrides applied at top environment instantiation.
parameters / parameteres	list(parameter_def_schema)	Bench parameter definitions. The 2023.4 reference table prints "parameteres" while its schema/example uses "parameters"; verify installed template.
veloce_ready	True False	Defaults True; enables emulation-ready generated code.
use_coemu_clk_rst_gen	True False	Defaults False; selects more capable co-emulation clock/reset generation.
clock_half_period	string	Half-period of generated top-level clock, e.g. 5ns.
clock_phase_offset	string	Delay before first clock edge.
reset_assertion_level	True False	Polarity of top-level reset driven by bench.
reset_duration	string	Initial reset assertion duration.
active_passive_default	ACTIVE PASSIVE	Default activity mode for all agents not explicitly listed; documented default is ACTIVE.
active_passive	list(active_passive_schema)	Per-BFM activity overrides.
interface_params	list(interface_param_schema)	Explicit BFM parameter values.
imports	list(import_schema)	Packages imported by bench package.
additional_tops	list(string)	Extra top-level modules instantiated in the bench.
mtlb_ready	True False	Defaults False; generate Matlab-oriented files.
use_bcr	True False	Defaults False; when True, make vrtn uses BCR underneath the flow.
bench_plusargs	list(plusarg_schema)	Capture/test command-line plusargs in generated test_top build phase.

10.3 active_passive_schema

Field	Meaning
bfm_name	BFM/agent path or name.
value	ACTIVE or PASSIVE.

For agents inside sub-environments, the 2023.4 reference says the full UVM path from the top environment is specified using underscores as delimiters because dots are reserved characters in Verilog.

10.4 interface_param_schema

Use interface_params to explicitly parameterize BFM. The 2023.4 reference says these entries are unnecessary when global elaborate_bfm_parameters is True and the generator can derive BFM parameter values automatically.

```
interface_params:
- bfm_name: "control_plane"
  value:
- name: "ADDR_WIDTH"
  value: "16"
- name: "DATA_WIDTH"
  value: "32"
```

10.5 plusarg_schema

Field	Required	Meaning
name	Yes	Plusarg name; alone it uses \$test\$plusargs and stores presence in a bit.
var_type	No*	SystemVerilog variable type for value-capturing plusargs.
plusarg_type	No*	Format such as %s, %d, %b, %h. var_type and plusarg_type are used together.
default_value	No	Value used when command-line plusarg is absent.
env_config	No	Top-environment configuration variable to assign from captured plusarg.

```
bench_plusargs:
- name: "scenario"
  var_type: "string"
  plusarg_type: "%s"
  default_value: "smoke"
- name: "enable_trace"
- name: "timeout_cycles"
  var_type: "int"
  plusarg_type: "%d"
env_config: "timeout_cycles"
```

10.6 Active/passive decisions

Agent role	Typical bench mode	Reason
Drives DUT request/input interface	ACTIVE	Driver/sequencer generates stimulus while monitor also observes.
Observes DUT response/output only	PASSIVE	Monitor publishes transactions; no stimulus driver needed.
Internal monitor reused at higher level	PASSIVE	Avoid multiple drivers on internal interface.
Responder model	ACTIVE	Responder sequence/driver may generate protocol responses.

11. Global YAML Structure

Global data applies across generated interfaces, environments, and benches. All global entries are optional and live under `uvmf: global:`.

11.1 Global schemas

Property	Default / purpose
header	Multi-line source header inserted into SystemVerilog files inheriting the base template. A YAML block scalar <code> </code> is recommended.
flat_output	False by default. True places generated source for each object in one flat directory instead of a <code>src</code> subdirectory.
vip_location	Default <code>verification_ip</code> . Parent directory for interface and environment generated code.
interface_location	Default <code>interface_packages</code> under <code>vip_location</code> .
environment_location	Default <code>environment_packages</code> under <code>vip_location</code> .
bench_location	Default <code>project_benches</code> .
elaborate_bfm_parameters	False if unspecified. True asks the generator to infer unspecified BFM parameter values from HVL-side settings; explicit <code>interface_params</code> remains the fallback for corner cases.

```
uvmf:
  global:
    header: |
      // Project verification code
      // Generated with UVMF YAML flow
    flat_output: "False"
    vip_location: "verification_ip"
    interface_location: "interface_packages"
    environment_location: "environment_packages"
    bench_location: "project_benches"
    elaborate_bfm_parameters: "True"
```

11.2 When to use automatic BFM parameter elaboration

Use `elaborate_bfm_parameters` when the HVL-side parameterization is straightforward and the generator can infer matching BFM values. If generation cannot determine a corner-case parameter expression correctly, set it to `False` and specify every BFM override explicitly under `bench interface_params`.

12. Multi-File Organization and Dependency Rules

12.1 Recommended file organization

```
yaml/
  global.yaml
  interfaces/
    req_if.yaml
    rsp_if.yaml
  components/
    block_predictor.yaml
  environments/
    block_env.yaml
  benches/
    block_tb.yaml
  block_tb_files.f
```

The UVMF schema does not require this exact directory layout; it is a practical organization that keeps definition types separate while still allowing one reproducible file list to build a bench.

12.2 Dependency order

Target being generated	Definitions that must be available to the generator run
Interface	Its interface YAML.
Utility component	Its util_components YAML.
Environment	Environment YAML plus referenced UVMF agents, utility components, and UVMF sub-environments.
Bench	Bench YAML plus top environment and recursively referenced interface/environment/component YAML.

12.3 One file versus many files

A single YAML file can contain several object definitions and global data. Multiple files are usually easier to review and reuse. The generator receives a combined data model from the files passed on the command line or through a file list.

Reuse practice: Keep interface definitions independent of DUT-specific bench details. Put DUT-specific active/passive mode, top clock/reset generation, DUT file lists, and top-level wiring in the bench layer or generated bench custom code.

13. From DUT Specification to YAML

A reliable UVMF YAML model starts with verification architecture, not with copying a template. The 2019.4 training flow begins by planning stimulus and checks, then creating protocol interfaces, the environment, the bench, completing the generated bench, and finally starting verification. The following procedure turns that flow into an engineering checklist.

13.1 Step 1 — Partition physical pins into protocol interfaces

- Group pins that participate in one transaction protocol and share clock/reset behavior.
- Separate unrelated input and output protocols into distinct interface packages when their transaction models differ.
- Identify whether each interface can act as initiator, responder, or monitor-only in the target bench.

13.2 Step 2 — Define transaction abstraction

- List untimed information required to reproduce one transfer: addresses, data, opcodes, strobes, IDs, status, etc.
- Do not put physical handshake timing in transaction fields unless it is intentional stimulus metadata.
- Mark randomizable fields with `isrand`; mark only semantically comparable fields with `iscompare`.

13.3 Step 3 — Define ports and parameters

- Map every BFM-visible pin to `port_schema`.
- Choose parameter names for widths that repeat across ports and transaction types.
- Write port direction from the testbench/BFM perspective.

13.4 Step 4 — Plan checks and analysis flow

- Determine which monitored input stream feeds a predictor/reference model.
- Determine the predicted transaction type.
- Select a built-in scoreboard when its ordering semantics fit; otherwise generate a custom scoreboard utility component.
- Add coverage utility components where transaction-level coverage belongs at environment scope.

13.5 Step 5 — Build environment hierarchy and connectivity

- Instantiate agents with stable instance names.
- Instantiate predictors/coverage/scoreboards.
- Connect driver endpoints to receiver endpoints using `tlm_connections`.
- Add sub-environments for vertical reuse rather than flattening all agents into one large environment.

13.6 Step 6 — Define bench policy

- Choose the top environment.
- Set each BFM ACTIVE or PASSIVE.
- Set clock and reset generation policy.
- Resolve BFM parameterization explicitly or with global automatic elaboration.
- Define bench plusargs that should feed top-level configuration.

13.7 Step 7 — Generate before custom coding

Generate the entire hierarchy and build it before inserting DUT-specific functionality. This separates YAML/schema problems from protocol/DUT problems and makes the first customization pass much faster.

14. Generated Output and Where to Customize

UVMF deliberately generates structural code and leaves protocol semantics, DUT instantiation, prediction, coverage, and custom checking to the engineer. The 2023.4 Users Guide identifies the main post-generation customization locations.

14.1 Interface package customization

Generated file / area	Typical customization
<if>_driver_bfm.sv	Implement protocol driving in <code>initiate_and_get_response()</code> for initiator behavior and/or <code>respond_and_wait_for_next_transfer()</code> for responder behavior.
<if>_monitor_bfm.sv	Implement <code>do_monitor</code> so BFM pins are sampled into transaction variables according to protocol timing.
<if>_responder_sequence.svh	Complete responder sequence body if the agent supports responder behavior.
<if>_transaction_coverage.svh	Add coverpoints, bins, crosses, and exclusions.
<if>2reg_adapter.svh	Implement <code>bus2reg()</code> and <code>reg2bus()</code> when using a UVM register model.
Additional sequence items / sequences	Extend generated transaction and sequence base classes; include new files in the package.

14.2 Environment package customization

Generated file / area	Typical customization
<env>_configuration.svh	Configure agents and environment policy required by DUT.
Predictor write functions	Implement prediction/reference-model algorithm.
Coverage utility component	Implement coverage model.
Custom scoreboard utility component	Implement storage, matching, comparison, and reporting.
Environment sequences	Add reusable environment-level sequences derived from generated sequence base.

14.3 Bench customization

Generated file / area	Typical customization
tb/testbench/hdl_top.sv	Instantiate the DUT and connect DUT ports to generated interface signal bundles/BFMs.
sim/Makefile	Add DUT Verilog/VHDL source file variables or build dependencies.
tb/sequences/..._bench_sequence_base.svh	Define typical virtual-sequence flow.
Additional bench sequences	Add scenario-specific virtual sequences derived from the bench sequence base.

Customization marker: Search generated output for `UVMF_CHANGE_ME`. The Users Guide uses that marker to identify expected manual implementation points. For regeneration-safe edits, keep source modifications inside the generator-provided custom pragma blocks.

15. Worked ALU-Style Example

This worked example follows the same architecture used by the supplied ALU training material—input agent, output agent, predictor, scoreboard, environment, and bench—but the YAML below is rewritten as a compact reference example rather than copied from the tutorial.

15.1 Input interface

```
uvmf:
  interfaces:
    alu_in:
      clock: clk
      reset: rst_n
      reset_assertion_level: "False"
      parameters:
        - name: "OPERAND_WIDTH"
          type: "int"
          value: "8"
      ports:
        - { name: "valid", width: "1", dir: "output" }
        - { name: "ready", width: "1", dir: "input" }
        - { name: "op", width: "3", dir: "output" }
        - { name: "a", width: "OPERAND_WIDTH", dir: "output" }
        - { name: "b", width: "OPERAND_WIDTH", dir: "output" }
      transaction_vars:
        - { name: "op", type: "bit [2:0]", isrand: "True", iscompare: "True" }
        - { name: "a", type: "bit [OPERAND_WIDTH-1:0]", isrand: "True", iscompare: "True" }
        - { name: "b", type: "bit [OPERAND_WIDTH-1:0]", isrand: "True", iscompare: "True" }
```

15.2 Output interface

```
uvmf:
  interfaces:
    alu_out:
      clock: clk
      reset: rst_n
      reset_assertion_level: "False"
      parameters:
        - name: "RESULT_WIDTH"
          type: "int"
          value: "16"
      ports:
        - { name: "done", width: "1", dir: "input" }
        - { name: "result", width: "RESULT_WIDTH", dir: "input" }
      transaction_vars:
        - { name: "result", type: "bit [RESULT_WIDTH-1:0]", isrand: "False", iscompare: "True" }
```

15.3 Predictor utility component

```
uvmf:
  util_components:
    alu_predictor:
      type: predictor
      analysis_exports:
        - name: "alu_in_ae"
          type: "alu_in_transaction"
      analysis_ports:
        - name: "predicted_ap"
          type: "alu_out_transaction"
```

15.4 Environment

```

uvmf:
  environments:
    alu_env:
      agents:
        - { name: "in_agent", type: "alu_in" }
        - { name: "out_agent", type: "alu_out" }
      analysis_components:
        - { name: "pred", type: "alu_predictor" }
      scoreboards:
        - name: "result_sb"
          sb_type: "uvmf_in_order_scoreboard"
          trans_type: "alu_out_transaction"
      tlm_connections:
        - driver: "in_agent.monitored_ap"
          receiver: "pred.alu_in_ae"
          validate: "True"
        - driver: "pred.predicted_ap"
          receiver: "result_sb.expected_analysis_export"
          validate: "True"
        - driver: "out_agent.monitored_ap"
          receiver: "result_sb.actual_analysis_export"
          validate: "True"

```

15.5 Bench

```

uvmf:
  benches:
    alu_tb:
      top_env: "alu_env"
      clock_half_period: "5ns"
      reset_assertion_level: "False"
      reset_duration: "200ns"
      active_passive:
        - { bfm_name: "in_agent", value: "ACTIVE" }
        - { bfm_name: "out_agent", value: "PASSIVE" }

```

15.6 Generation command

```

python $UVMF_HOME/scripts/yaml2uvmf.py \
  alu_in.yaml alu_out.yaml alu_predictor.yaml alu_env.yaml alu_tb.yaml

```

After generation, implement the input driver/monitor protocol timing, output monitor, predictor algorithm, DUT instance and connections, then compile/run. During YAML iteration, regenerate cleanly; after hand edits, use the merge flow.

16. Practical AXI4-Stream-Style Interface Template

Practical supplement: This section is an engineer-authored example using the UVMF schema patterns described in the supplied manuals. It is not a canonical Siemens/Mentor AXI4-Stream template. Adapt widths, optional sideband signals, transaction semantics, and BFM timing to the DUT and installed UVMF release.

The example below models a testbench initiator: TDATA/TVALID and sideband payload are outputs from the testbench BFM, while TREADY is an input observed from the DUT. transfer_delay is stimulus timing metadata and is therefore excluded from transaction comparison.

```

uvmf:
  interfaces:
    axi4stream:
      clock: ACLK
      reset: ARESETn
      reset_assertion_level: "False"
      vip_lib_env_variable: UVMF_VIP_LIBRARY_HOME
      veloce_ready: "True"

  parameters:
    - { name: "AXI4S_DATA_WIDTH", type: "int", value: "64" }
    - { name: "AXI4S_ID_WIDTH", type: "int", value: "1" }
    - { name: "AXI4S_DEST_WIDTH", type: "int", value: "1" }
    - { name: "AXI4S_USER_WIDTH", type: "int", value: "1" }

  ports:
    - { name: "TVALID", width: "1", dir: "output" }
    - { name: "TREADY", width: "1", dir: "input" }
    - { name: "TDATA", width: "AXI4S_DATA_WIDTH", dir: "output" }
    - { name: "TKEEP", width: "AXI4S_DATA_WIDTH/8", dir: "output" }
    - { name: "TSTRB", width: "AXI4S_DATA_WIDTH/8", dir: "output" }
    - { name: "TLAST", width: "1", dir: "output" }
    - { name: "TID", width: "AXI4S_ID_WIDTH", dir: "output" }
    - { name: "TDEST", width: "AXI4S_DEST_WIDTH", dir: "output" }
    - { name: "TUSER", width: "AXI4S_USER_WIDTH", dir: "output" }

  transaction_vars:
    - { name: "data", type: "bit [AXI4S_DATA_WIDTH-1:0]", isrand: "True", iscompare: "True" }
    - { name: "keep", type: "bit [(AXI4S_DATA_WIDTH/8)-1:0]", isrand: "True", iscompare: "True" }
    - { name: "strb", type: "bit [(AXI4S_DATA_WIDTH/8)-1:0]", isrand: "True", iscompare: "True" }
    - { name: "last", type: "bit", isrand: "True", iscompare: "True" }
    - { name: "id", type: "bit [AXI4S_ID_WIDTH-1:0]", isrand: "True", iscompare: "True" }
    - { name: "dest", type: "bit [AXI4S_DEST_WIDTH-1:0]", isrand: "True", iscompare: "True" }
    - { name: "user", type: "bit [AXI4S_USER_WIDTH-1:0]", isrand: "True", iscompare: "True" }
    - { name: "transfer_delay", type: "int unsigned", isrand: "True", iscompare: "False" }

```

16.1 What the YAML does not implement

- TVALID/TREADY handshake timing.
- Stable-payload requirements while TVALID is asserted and TREADY is deasserted.
- Packet boundary behavior using TLAST.
- Responder/backpressure policy.
- Driver and monitor sampling clocking behavior.
- Protocol-specific constraints among TKEEP, TSTRB, TLAST, or optional sideband signals.

Those behaviors belong in the generated driver BFM, monitor BFM, sequences, and protocol constraints after the structural YAML has been generated successfully.

17. Validation, Debugging, and Common Errors

17.1 Validate in layers

Layer	Question	Recommended action
1 — YAML syntax	Can the YAML parser read the mapping/list structure?	Fix indentation, list markers, quoting, and duplicate/misplaced keys.
2 — UVMF schema	Does yaml2uvmf.py accept the expected properties and referenced definitions?	Run generator in a clean destination and read generator diagnostics.
3 — Generated compile	Does generated infrastructure compile before hand edits?	Run the generated compile/cli flow immediately.
4 — Connectivity	Do analysis ports/exports resolve and does data flow reach scoreboards?	Use validate where supported and inspect generated connection code.
5 — Protocol behavior	Do driver/monitor BFM's obey the DUT protocol?	Add protocol-specific implementation and focused sequences.
6 — Functional checking	Do prediction, scoreboards, coverage, and register-model paths behave correctly?	Add expected model/checking incrementally.

17.2 Common failure patterns

Symptom	Likely cause / next check
Generator says referenced type is unknown	Referenced interface/utility/sub-environment YAML was not passed to this run, or type spelling does not match definition name.
YAML change appears to have no effect	Existing output was skipped. Generate into a clean directory, use overwrite intentionally, or use merge flow after hand edits exist.
Driver appears reversed	Port direction was written from DUT perspective instead of testbench/BFM perspective.
Scoreboard has extra transactions	Generated predictor placeholder behavior can broadcast transactions from each input until prediction logic is implemented; inspect analysis connections and predictor write functions.
Scoreboard never receives actuals	The agent monitored_ap is not connected to actual_analysis_export, endpoint path is misspelled, or monitor BFM is not producing transactions.
Environment config value fails	config_variable_values path is wrong or value type is incompatible.
Sub-environment active/passive setting not applied	Hierarchical BFM path does not use the required underscore-delimited form described by the bench schema.
Reg model connection fails	Adapter/interface type/package/class does not match generated environment; QVIP/custom adapter integration may require UVMF_CHANGE_ME edits.
Merge succeeded but compile fails	Structural YAML changed a generated API that hand-written custom code still references.
Manual code disappeared after merge/regeneration	Edit was outside a recognized UVMF custom pragma block.
DPI link/build failure	C source list, compile flags, link flags, symbol signature, or shared-object settings do not match the build.
AXI4-Stream monitor compares transfer_delay	Metadata field should normally use iscompare: "False".

17.3 Debugging merge problems

- Run with `--merge_debug` and compare the pure-new generation with the merged tree.
- Run with `--merge_verbose` to identify which files and labeled blocks were matched.
- If a custom block is reported missing, decide whether the new template intentionally removed it before using `--merge_skip_missing_blocks`.
- Compile after every structural merge before making additional manual edits.

17.4 Release-sensitive checks

- Run `yaml2uvmf.py --version` and `--help` before relying on a command-line switch from documentation.
- Compare your installed `new_interface.yaml` / `new_environment.yaml` templates with this guide before introducing optional keys.

- Treat QVIP agent names, AP keys, and transaction types as QVIP-release-specific.
- Treat legacy 2019.4 keys described in Appendix C as examples of workflow, not automatically valid 2023.4 schema.

18. Quick Reference Checklists

18.1 New interface checklist

Check	Done
Clock/reset names and reset polarity defined.	☐
All interface pins listed; direction verified from TB/BFM perspective.	☐
Widths parameterized where appropriate.	☐
Transaction fields represent untimed protocol information.	☐
isrand and iscompare reviewed per field.	☐
Constraints added only where structurally appropriate.	☐
Configuration variables separated from transaction variables.	☐
Generator runs in clean destination.	☐
Generated driver/monitor compile before custom implementation.	☐

18.2 New environment checklist

Check	Done
Every referenced agent type has YAML in the generation run.	☐
Predictor/coverage/custom scoreboard definitions are available.	☐
Scoreboard type and transaction type selected.	☐
Expected and actual transaction paths are defined.	☐
TLM driver/receiver endpoint names validated.	☐
Sub-environment reuse and parameter overrides planned.	☐
Configuration variable defaults/paths reviewed.	☐
Register model and QVIP paths reviewed if used.	☐

18.3 New bench checklist

Check	Done
top_env name exactly matches environment definition.	☐
Every BFM active/passive policy is intentional.	☐
Clock half period, phase offset, reset polarity, and reset duration reviewed.	☐
BFM parameterization is explicit or global auto-elaboration is enabled.	☐
Bench plusargs are typed and mapped to configuration where required.	☐
DUT is instantiated and connected in hdl_top custom area.	☐
DUT source files are included in simulation build flow.	☐
Generated bench compiles before tests are expanded.	☐

18.4 Safe regeneration checklist

Check	Done
Working tree backed up/committed.	☐
Manual edits are inside recognized UVMF custom pragma blocks.	☐

Check	Done
YAML structural change reviewed for API impact.	☐
--merge_source points at correct source tree.	☐
--merge_debug/--merge_verbose used when change is non-trivial.	☐
Post-merge compile completed before new manual edits.	☐

19. YAML Test List Format for UVMF VRM

UVMF uses a second YAML structure for regression execution with the Verification Run Manager (VRM). This is distinct from the YAML consumed by `yaml2uvmf.py`. When a VRM test-list file has a `.yaml` extension, VRM treats it as a YAML test list rather than the standard ASCII test-list format. The YAML form is more verbose, but it can use descriptive test names independently of the actual UVM test class and can attach build, run, seed, include, and per-test runtime information in a structured form.

Source basis for Chapter 19: UVM Framework Users Guide, Version 2023.4, Section 4.2.4 (YAML Test List Format), Section 4.2.4.1 (YAML Test List Schema), Section 4.2.4.2 (YAML Test List Entry Descriptions), and Section 4.3 (Operation and Results).

Important distinction: UVMF code-generator configuration files use the root key `'uvmf:'`. A VRM YAML regression list uses the root key `'uvmf_testlist:'`. The VRM test list describes what to build and run; it is not an input file for `yaml2uvmf.py`.

19.1 Canonical YAML test-list schema

```
uvmf_testlist:
  testbenches:
    - name: "testbench_name"          # Required
      extra_build_options: "extra_build_options" # Optional
      extra_run_options: "extra_run_options" # Optional
      symlinks:                        # Optional
        - [target_file1, link_name1]
        - [target_file2, link_name2]

  tests:
    - testbench: "testbench_name"      # Must be specified once
      name: "test_name"                # Required for each test
      repeat: 1                        # Optional; defaults to 1
      seeds: [12345, 67890]            # Optional; random if omitted
      uvm_testname: "uvm_test_name"    # Optional; defaults to test name
      extra_args: "+ARG=VALUE"         # Optional per-test runtime args

  include:                             # Optional
    - "path_to_included_testlist_file_1"
    - "path_to_included_testlist_file_2"
```

19.2 Entry reference

Key	Required?	Meaning and practical behavior
<code>uvmf_testlist</code>	Yes	Root mapping for a VRM YAML test list. This is different from the <code>'uvmf:'</code> code-generator root.
<code>testbenches</code>	Yes in practice	List of testbench definitions that VRM can build and use for tests.
<code>testbenches[].name</code>	Yes	Testbench name. Each testbench list entry must have a name.
<code>extra_build_options</code>	No	Additional command-line arguments applied to the build step for that testbench.
<code>extra_run_options</code>	No	Additional command-line arguments applied to the run step for that testbench.
<code>symlinks</code>	No	List of <code>[target_file, link_name]</code> pairs created for each test instance. Relative target paths are based on the VRM invocation directory.
<code>tests</code>	Yes in practice	Ordered list of testbench-selection and test entries. Tests use the most recently specified testbench.
<code>tests[].testbench</code>	Once before tests	Selects the testbench for this and subsequent test entries until another testbench is specified.
<code>tests[].name</code>	For each actual test	VRM-visible test name. It can be an arbitrary descriptive string when <code>uvm_testname</code> is specified.
<code>repeat</code>	No	Number of iterations. Defaults to 1.
<code>seeds</code>	No	Integer seed list for iterations. Omitted seeds are random; if fewer seeds than iterations are supplied, the remaining seeds are generated randomly.
<code>uvm_testname</code>	No	Actual UVM test class name. Defaults to the value of <code>name</code> when omitted.
<code>extra_args</code>	No	Additional runtime arguments for this test entry.

Key	Required?	Meaning and practical behavior
include	No	List of additional YAML test-list files to parse. Relative include paths are resolved from the parent test-list directory.

Precision note: The 2023.4 schema key is 'seeds' (plural). The accompanying entry-description text labels the behavior as 'seed', but describes a list of integer seed values.

19.3 Minimal working test list

A minimal YAML regression list needs a testbench definition and at least one named test. If `uvm_testname` is omitted, VRM assumes that the test name is also the UVM test class name.

```
uvmf_testlist:
  testbenches:
    - name: "my_bench"

  tests:
    - testbench: "my_bench"
      name: "smoke_test"
```

19.4 Practical regression example

The example below shows a practical pattern: one bench, a deterministic smoke run, a repeated random run that inherits the previously selected testbench, a descriptive VRM test name mapped to a UVM class, per-test plusargs, a symbolic link, and an included test-list fragment.

```
uvmf_testlist:
  testbenches:
    - name: "axi4stream_bench"
      extra_build_options: "USE_64BIT=1"
      extra_run_options: "-uvmcontrol=all"
      symlinks:
        - ["../vectors/input_packets.hex", "input_packets.hex"]

  tests:
    - testbench: "axi4stream_bench"
      name: "smoke_20_packets"
      repeat: 1
      seeds: [12345]
      uvm_testname: "axi4stream_smoke_test"
      extra_args: "+NUM_PKTS=20"

    - name: "random_stress_1000_packets"
      repeat: 5
      uvm_testname: "axi4stream_random_test"
      extra_args: "+NUM_PKTS=1000"

  include:
    - "nightly/error_injection.yaml"
```

19.5 Testbench selection and inheritance

- A test-list entry can specify a testbench, a test name, or both.
- A named test cannot appear before a testbench has been selected; VRM reports an error in that case.
- After a testbench is selected, following tests execute against that same testbench until another testbench name is specified.
- Therefore, the testbench key does not need to be repeated on every test entry when a group of tests targets the same bench.

19.6 Repeat counts and seed handling

- repeat is optional and defaults to 1.
- seeds is optional. If omitted, each iteration receives a random seed.
- If the supplied seeds list is shorter than the repeat count, VRM fills the remaining iterations with random seeds.
- For reproducible debugging, record the seed associated with a failing iteration and rerun that same UVM test with the same runtime arguments.

19.7 Descriptive names versus UVM test classes

Without `uvm_testname`, VRM assumes that name is the UVM test name. With `uvm_testname`, the name field becomes a user-controlled label for the regression entry. This allows the same UVM test class to appear multiple times with different configurations while keeping the VRM output directories easy to identify.

```
tests:
- testbench: "axi4stream_bench"
  name: "short_packets_ready_always"
  uvm_testname: "axi4stream_random_test"
  extra_args: "+PKT_MIN=1 +PKT_MAX=8 +READY_MODE=ALWAYS"

- name: "long_packets_random_backpressure"
  uvm_testname: "axi4stream_random_test"
  extra_args: "+PKT_MIN=64 +PKT_MAX=512 +READY_MODE=RANDOM"
```

19.8 Includes and symbolic links

Feature	Path rule	Practical use
include	Relative paths are resolved relative to the parent YAML test-list file.	Split smoke, nightly, protocol-error, and feature suites into maintainable files.
symlinks target	A relative target path is resolved relative to the directory where VRM was invoked.	Make vectors, configuration files, models, or data files visible inside each test run directory.
symlinks link_name	Created in each test instance run directory; a path inside link_name is unsupported.	Use a simple filename for the link itself.

19.9 VRM execution and result layout

After parsing the test list, VRM builds the selected benches and schedules their simulations. The 2023.4 User Guide describes bench builds running in parallel, followed by parallel simulations for each successfully built bench, UCDB merging as runs complete, and generation of an HTML coverage report after all simulations finish.

```
./sim/VRMDATA/
top/
  each_top~<bench_name>/
    build_group/
      build_task/
        run_fork/
          run~<bench_name>-<test_name>-<iteration>-<seed>/
```

If a bench build or simulation fails, inspect the `execScript.log` file beneath the corresponding build or run directory.

19.10 Practical review checklist

- Use the root key `uvmf_testlist:`, not `uvmf:`.
- Use a .yaml test-list filename when invoking the YAML VRM test-list flow.
- Declare each testbench under `testbenches:` with a unique name.
- Select a testbench before the first named test.
- Set repeat explicitly when multiple iterations are required.
- Provide seeds for deterministic reproduction, or allow VRM to generate random seeds.
- Use `uvm_testname` when the human-readable regression name should differ from the UVM class name.
- Put per-test plusargs and runtime controls in `extra_args`.
- Use `include` to compose larger regression suites from smaller YAML files.

Appendix A - Complete YAML Skeletons

A.1 Interface skeleton

```
uvmf:
  interfaces:
    <interface_name>:
      clock: <clock>
      reset: <reset>
      reset_assertion_level: "True|False"
      vip_lib_env_variable: <env_var>
      veloce_ready: "True|False"
      mtlb_ready: "True|False"
```

```

imports: []
parameters: []
hdl_typedefs: []
hvl_typedefs: []
ports: []
transaction_vars: []
transaction_constraints: []
config_vars: []
config_constraints: []
# dpi_define: ...

```

A.2 Utility component skeleton

```

uvmf:
  util_components:
    <component_name>:
      type: predictor|coverage|scoreboard
      analysis_exports: []
      analysis_ports: []
      qvip_analysis_exports: []
      parameters: []

```

A.3 Environment skeleton

```

uvmf:
  environments:
    <env_name>:
      agents: []
      non_uvmf_components: []
      qvip_memory_agents: []
      parameters: []
      hvl_pkg_parameters: []
      imports: []
      analysis_components: []
      scoreboards: []
      subenvs: []
      qvip_subenvs: []
      analysis_ports: []
      analysis_exports: []
      tlm_connections: []
      qvip_connections: []
      config_vars: []
      config_variable_values: []
      config_constraints: []
      imp_decls: []
      # register_model: ...
      # dpi_define: ...
      typedefs: []
      mtlb_ready: "False"

```

A.4 Bench skeleton

```

uvmf:
  benches:
    <bench_name>:
      top_env: <env_name>
      top_env_params: []
      parameters: []
      veloce_ready: "True"
      use_coemu_clk_rst_gen: "False"
      clock_half_period: "5ns"
      clock_phase_offset: "0ns"
      reset_assertion_level: "False"
      reset_duration: "100ns"
      active_passive_default: "ACTIVE"
      active_passive: []
      interface_params: []
      imports: []
      additional_tops: []
      mtlb_ready: "False"
      use_bcr: "False"
      bench_plusargs: []

```

A.5 Global skeleton

```

uvmf:
  global:
    header: |
      // common generated-file header
    flat_output: "False"
    vip_location: "verification_ip"
    interface_location: "interface_packages"
    environment_location: "environment_packages"
    bench_location: "project_benches"
    elaborate_bfm_parameters: "False"

```

A.6 VRM YAML test-list skeleton

```

uvmf_testlist:
  testbenches:
    - name: "<bench_name>"
      extra_build_options: "<optional_build_options>"
      extra_run_options: "<optional_run_options>"
      symlinks:
        - ["<target_file>", "<link_name>"]

  tests:
    - testbench: "<bench_name>"
      name: "<regression_entry_name>"
      repeat: 1
      seeds: [12345]
      uvm_testname: "<uvm_test_class>"
      extra_args: "<optional_runtime_args>"

  include:
    - "<optional_nested_testlist.yaml>"

```

Appendix B - Schema Quick Reference

B.1 Interface schemas

Schema	Required fields	Optional fields / defaults
import_schema	name	—
parameter_def_schema	name, type	value
typedef_schema	name, type	—
port_schema	name, width, dir	reset_value (default 0)
transaction_schema	name, type	isrand=False, iscompare=True, unpacked_dimension, comment
config_var_schema	name, type	isrand=False, value, comment
constraint_schema	name, value	comment
dpi_schema	name, files, comp_args, link_args	imports; exports are documented but unsupported in 2023.4
dpi_import_schema	name, return_type, c_args, sv_args	sv_arg unpacked_dimension

B.2 Utility component schemas

Schema	Fields
analysis_schema	name, type
parameter_def_schema	name, type, optional value

B.3 Environment schemas

Schema	Key fields
component_schema	name, type, extdef, optional parameters; specialized examples may add qvip_environment or reg_block_instance_name
scoreboard_schema	name, sb_type, trans_type, optional parameters
parameter_use_schema	name, value
parameter_def_schema	name, type, optional value
import_schema	name
qvip_env_schema	name, type
tlm_port_schema	name, trans_type, connected_to
tlm_schema	driver, receiver, optional validate
qvip_tlm_schema	driver, ap_key, receiver, optional validate
config_var_schema	name, type, optional isrand/value/comment
config_value_schema	name, value
constraint_schema	name, value, optional comment
imp_decl_schema	name
reg_model_schema	use_adapter, use_explicit_prediction, package/block/adaptor fields, maps[]
typedef_schema	name, type

B.4 Bench schemas

Schema	Key fields
parameter_use_schema	name, value
parameter_def_schema	name, type, optional value

Schema	Key fields
import_schema	name
active_passive_schema	bfm_name, value=ACTIVE PASSIVE
interface_param_schema	bfm_name, value=[parameter_use_schema]
plusarg_schema	name; optional var_type, plusarg_type, default_value, env_config

B.5 VRM YAML test-list keys

Key	Purpose
uvmf_testlist	Root of the VRM YAML test-list document.
testbenches	Defines benches and optional build/run options and symlinks.
tests	Defines testbench selection, named regression entries, repeats, seeds, UVM test mapping, and per-test runtime arguments.
include	Includes additional YAML test-list files; relative paths are resolved from the parent test-list directory.

Appendix C - Release and Documentation Notes

C.1 2023.4 is the primary schema basis

The formal property/schema tables in this guide track the supplied UVMF YAML Reference Manual Version 2023.4. The supplied UVM Framework Users Guide Version 2023.4 is used for generator workflow and generated-file customization responsibilities.

C.2 2019.4 tutorial material is used for practical flow

The ALU tutorial and student workbook provide a useful end-to-end engineering sequence and concrete examples of port direction, transaction abstraction, TLM connections, predictor/scoreboard topology, and bench active/passive decisions. Some YAML details in those older materials should not be assumed to be current schema without checking the installed release.

C.3 Legacy/example keys to verify before use

Item	Why to verify
response_info	Shown in the 2019.4 ALU tutorial. It is not described in the supplied 2023.4 YAML Reference interface schema chapter.
transaction_constraints spelling	One 2019.4 tutorial slide text shows this misspelling. The 2023.4 schema name is transaction_constraints / constraint_schema.
Bench parameteres spelling	The supplied 2023.4 YAML Reference bench property table prints parameteres, while parameter_def_schema examples use parameters. Verify the installed template/help.
--generate short switch	The 2023.4 YAML Reference uses -g; the 2023.4 Users Guide table appears to show -f, which is also used for --file. Use yaml2uvmf.py --help.

C.4 Practical authority order

13. Installed yaml2uvmf.py --help / --version.
14. Installed new_interface.yaml, new_environment.yaml, and example YAML files for that release.
15. UVMF YAML Reference Manual matching the installed release.
16. UVM Framework Users Guide matching the installed release.
17. Older tutorials/workbooks for workflow concepts and examples.

Appendix D - Glossary

Term	Meaning in this guide
Agent	UVMF protocol component generated from an interface YAML definition; generally includes sequencer/driver/monitor/configuration infrastructure.
BFM	Bus Functional Model. Generated HDL-side driver/monitor modules/interfaces used for protocol pin-level behavior.
Bench	Top-level DUT-specific UVMF assembly containing top environment, tests, virtual sequence infrastructure, hdl_top/hvl_top, and simulation support.
Environment	Reusable composition of agents, analysis components, scoreboards, sub-environments, configuration, and TLM connectivity.
Interface YAML	Generator description for reusable protocol verification IP: clock/reset, pins, transaction fields, configuration, etc.
monitored_ap	Built-in analysis port on a UVMF agent used to broadcast monitored transactions.
Predictor	Analysis component that receives observed stimulus and produces expected transactions/results.
Scoreboard	Checker comparing expected and actual transaction streams. Built-in UVMF scoreboards expose expected_analysis_export and actual_analysis_export.
TLM connection	Transaction-level connection from an emitting analysis port to a receiving export/imp.
Utility component	Generator-defined predictor, coverage component, or custom scoreboard used inside an environment.
UVMF_CHANGE_ME	Marker used in generated output to identify intended manual implementation points.
UVMF custom pragma block	Labeled generated-code region designed to preserve hand-written code across merge/regeneration.
QVIP	Questa Verification IP integration; UVMF can instantiate Configurator-generated sub-environments and connect QVIP analysis streams.

Appendix E - Source Documents Used

This guide was authored from the following user-supplied documents. It paraphrases their technical content and uses rewritten examples so that the result functions as a practical engineering desk reference rather than a reproduction of the original manuals.

Source	Version / role
UVMF YAML Reference Manual	Version 2023.4 — primary YAML schema and generator/merge reference.
UVM Framework Users Guide	Version 2023.4 — code-generator workflow and generated-code customization reference.
UVM Framework ALU Tutorial: A Step By Step Guide	UVMF 2019.4 — practical interface/environment/bench decomposition and end-to-end example flow.
UVM Framework Student Workbook	2019.4 — practical course flow and generation workflow context.

Release verification: Before adopting any example directly, compare it with the templates and script help shipped with the UVMF release installed in your project environment.